

Tokenized Compute Markets: A Self-Growing Mechanism for Community- Owned, Verifiable, Confidential, and Agentic AI Compute

Avi Gaba

Independent Researcher · EventHorizon Labs / Singularity Layer

Working Paper · contact: work@ehlabs.xyz

Abstract. AI inference is becoming a foundational economic resource, yet its supply is concentrated: the three largest hyperscalers — AWS, Microsoft Azure, and Google Cloud — captured nearly 71% of the global cloud infrastructure (IaaS) services market in 2024.^[12] This concentration keeps the cost of intelligence high and its availability gated by a few operators. We present **Tokenized Compute Markets (TCM)**, a mechanism coupling capital formation, physical infrastructure deployment, and AI inference into a single *self-growing* economic loop, in which trading activity itself finances the acquisition of new machines, so the network expands its own compute capacity without central capital allocation. Any community can launch a *Grid Cluster*, a branded group of compute providers coordinated by a cluster token (throughout, “token” denotes an on-chain crypto asset, an SPL token on Solana, not the text tokens processed during LLM inference). Inference is priced in a stable unit of account (USD) but settled in the cluster token; a swap-fee levy on the cluster's canonical liquidity pool accrues to a *Hardware Fund* that autonomously provisions new machines (consumer devices for a low-cost open tier, or TEE-verified GPUs for a confidential tier) as funding targets are met, gated on externally-paid inference so that capacity tracks real demand rather than self-trading. We give the formal mechanics of the loop, an incentive analysis of the actors, a security model that separates *integrity* (enforced by tolerance-checked audit re-execution and stake slashing on the open tier, and by enclave attestation on the confidential tier) from *confidentiality* (hardware-enforced via Trusted Execution Environments), and an illustrative capacity model showing how a cluster reaching a \$1B token market capitalization could fund on the order of 10^3 owned confidential GPUs or 10^4 consumer nodes. Every role in the market (consumer, node provider, and cluster owner) is agent-eligible, so autonomous agents can pay for, provide, and coordinate compute without a human in each loop. We are explicit about the failure modes (token velocity, demand–supply causality, and liquidity fragmentation) and specify the constraints under which the loop remains sound. This is a design (pre-deployment) paper; all quantitative figures are illustrative parameterizations, not measurements.

1. Introduction

Every application, agent, and workflow that uses a large language model (LLM) consumes *inference*: the forward pass that turns a prompt into a response. As agentic software proliferates, inference demand is set to grow faster than any single vendor can serve neutrally. Today that supply is centralized. A decentralized alternative faces a bootstrapping problem familiar to all Decentralized Physical Infrastructure Networks (DePIN): supply (machines) and demand (paying inference) must grow together, but neither arrives first without incentive.

The dominant DePIN pattern (emit a token to subsidize supply and hope demand follows) routinely fails, because it inflates capacity that has no revenue to sustain it. TCM is designed to address this directly by making *trading activity itself* deploy real, revenue-capable infrastructure,

while keeping the pricing of the underlying service insulated from token volatility. Our contributions are:

- **A mechanism** (§3) in which communities launch branded compute clusters whose token natively couples capital formation to hardware deployment via a Hardware Fund.
- **A pricing rule** (§4) that quotes inference in a stable unit (USD) but settles in the cluster token, protecting node-operator revenue from volatility, and a precise account of *who* actually experiences “cheaper inference” and who does not.
- **A two-tier security model** (§5) separating verifiable integrity (tolerance-checked audit re-execution and stake slashing on the open tier; enclave attestation on the confidential tier) from confidentiality (hardware-enforced by a TEE), with the audit-economics condition under which the slashing deterrent holds.
- **An incentive and capacity analysis** (§7–8) with an illustrative \$1B worked example, and an explicit statement of failure modes and the constraints that keep the loop sound (§10).

2. Background and Related Work

Confidential inference. When inference runs on hardware the data owner does not control, the operator can read the prompt. This is close to unavoidable in the plaintext setting: hidden states carry enough information to reconstruct inputs, and text embeddings reveal nearly as much as the text itself.^[1] Three research families address this. *Fully Homomorphic Encryption* (FHE) computes on ciphertext;^[2] strongest guarantee, but orders of magnitude too slow for real-time LLMs because non-polynomial operations (softmax, GELU, layernorm) are extremely expensive under encryption. *Secure Multi-Party Computation* (MPC) secret-shares the computation across non-colluding parties;^[3] faster than FHE but still $\sim 1000\times$ slower than plaintext and communication-heavy. *Permutation/obfuscation* schemes^[4,5] shuffle hidden states for near-plaintext speed, but a 2025 attack reconstructs prompts from permuted states with near-perfect accuracy *because the adversary holds the model weights*;^[6] their true security rests not on the permutation but on a non-collusion assumption between parties. A recent survey catalogs the field,^[7] and distributed approaches such as Fission^[8] explore splitting inference across machines.

Trusted Execution Environments. A TEE is a hardware-isolated region (Intel TDX, AMD SEV-SNP, NVIDIA H100/H200 confidential mode) whose memory is encrypted by the silicon and inaccessible to the machine's operator, and which can emit a signed *attestation* proving genuine hardware, confidential mode, and the exact code/model hash. Benchmarks place the overhead of H100 confidential computing below $\sim 5\%$ for typical LLM queries: the cost is data transfer across the bus, not computation^[9], and CPU TEEs under $\sim 10\%$ throughput.^[10] TEE attestation is therefore the practical basis for *both* confidentiality and verifiability today.

Positioning. TCM does not propose a new cryptographic primitive for confidential inference; it adopts TEE attestation as the confidentiality mechanism and contributes the *economic and coordination layer*: how community-owned compute is financed, deployed, priced, and secured against integrity faults. Where prior decentralized-inference systems assume the machines exist, TCM specifies how trading finances their acquisition.

Relation to existing DePIN compute networks. Decentralized-compute markets can be distinguished by *how new hardware is financed*. Emission-subsidized designs pay providers in newly minted tokens to attract supply, which grows capacity ahead of (and often independent of) revenue. On-chain-matching designs coordinate buyers and sellers of compute trustlessly but presuppose that the machines already exist; they route demand to standing supply rather than

creating supply. Marketplace/aggregator designs pay existing operators for utilization, again matching rather than financing. TCM occupies a distinct point: it is the only pattern, to our knowledge, in which *trading activity on a cluster's own token directly finances the acquisition of new revenue-capable hardware* through the Hardware Fund, coupling capital formation to market activity rather than to token emissions or to pre-existing capacity. This is a claim about mechanism structure, not a benchmark comparison; a quantitative head-to-head against specific networks is out of scope here and deferred to an evaluation of a deployed system.

3. The Mechanism

Actors. (i) *Node providers* contribute machines and serve inference; (ii) *consumers* (human or agent) pay for inference; (iii) *cluster owners* launch and parameterize a cluster; (iv) the *protocol* (Singularity Layer) operates *verifiable* orchestration, attestation verification, the staking/slashing engine, and the Hardware Fund — orchestration is not a trusted operator service: each node signs job assignments and results with its own key, and the protocol anchors these signatures on a Solana program, so that any party can independently verify that work was routed to and served by a legitimately staked node (see §5); and (v) a *base protocol token* that secures the network (via provider staking) and captures protocol-level value. Crucially, every actor role is *agent-eligible*: an autonomous agent can be a consumer (paying for its own inference), a node provider (staking and serving compute from the machine it runs on), and a cluster owner (launching and parameterizing a cluster). The mechanism is thus *agentic* by construction: it is designed for a population of software agents that transact, provide, and coordinate compute without a human in each loop.

Grid and staking. The base layer is the Grid: a decentralized network of machines (consumer devices and TEE servers). A provider joins by staking a fixed bond of $S = 50,000$ SGL (the base protocol token), an economic security bond that is identical for every node and is not rescaled as the token's price moves. A flat, nominal stake keeps the requirement simple and predictable and makes the security bond a fixed, unforgeable commitment per node rather than a moving target. This is a deliberate tradeoff: a nominal bond denominates the cost-to-attack in the token, so the USD-value of the security guarantee falls if the token depreciates; the protocol accepts this for launch simplicity and can revisit a USD-targeted or dynamically-scaled bond if attack economics warrant. Providers are matched to jobs by an orchestrator combining round-robin fairness with reputation-weighted priority.

Grid Clusters. A cluster is a group of providers under one brand, one community, and one token. Anyone may launch one (a startup, a DAO, a creator, or an individual) and admit members publicly, privately (invite links/codes), or in a mixed mode. At launch (and only then; parameters are fixed pre-launch to prevent redefinition after holders commit) the owner sets: the cluster token fee and its routing (to node providers and/or the Hardware Fund); the fund's target hardware type: a Mac node (Singularity-hosted, or shipped to the operator) or a cloud/TEE machine; and the access mode.

Fee flow. For each served request, 80% of the inference fee goes to the node that served it. The remaining share supports base-token buybacks, staking rewards, and protocol revenue via the protocol staking engine. The cluster-token fee is routed per the owner's configuration.

Token launch. Cluster tokens launch on open bonding markets (via the bags.fm API at launch, with pump.fun and, if required, a custom market to follow), so a market forms openly with no private primary sale. Concretely, a cluster token is a **classic SPL token on Solana**. Because a classic SPL token carries no token-level transfer fee, the trade levy (below) is enforced at the liquidity-pool layer (via the swap-fee configuration of the Meteora Dynamic AMM v2 (DAMM v2)

pool set at launch) rather than by the token program itself. This is a deliberate tradeoff: a pool-layer fee applies only to swaps through the canonical pool, so wallet-to-wallet transfers, OTC deals, and any competing pool are not taxed, and arbitrage between a taxed and an untaxed pool can leak value; the levy is therefore best read as “a swap fee on the canonical pool,” not a levy on literally every transfer. A protocol-wide alternative exists — the SPL **Token-2022** transfer-fee extension enforces a fee on every transfer at the token-program level — but it is not universally supported by launchpads and routers today, so we choose classic SPL for launch-time liquidity and compatibility and route canonical volume to the taxed pool via incentives. Throughout this paper “token” refers to this on-chain asset, distinct from the *text tokens* that are the unit of LLM inference.

The Hardware Fund. This is the core novelty. A configurable portion of the cluster's token activity accrues to a Hardware Fund. Concretely, a transfer levy τ (illustratively 2%) applies to each buy and sell; the owner routes a fraction ρ of that levy to the fund. When the fund balance reaches the cost C_h of the owner-selected machine type, the protocol *autonomously acquires that machine* (buying or renting per configuration) and adds it to the cluster. If bought, the protocol manages procurement and maintenance, using subsequent target hits to fund upkeep. This makes the network *self-growing*: a cluster expands not only financially but *physically*, as activity $\rightarrow$ more fund $\rightarrow$ more machines $\rightarrow$ more capacity, with no central capital-allocation decision at any step. To keep that loop tied to genuine demand rather than manufactured turnover, the fund is *gated on externally-paid inference*: fund accrual and a cluster's reputation/visibility are driven by swap fees and inference revenue *net of the cluster owner's own wallets*, so self-trading a token to hit hardware targets is screened out (see §7 and F5 in §10). Capacity therefore grows toward compute the market is actually paying to use.

3.1 A Permissionless Market-of-Markets

A defining property is that autonomous agents are first-class on *every* side of the market, not merely as buyers. On the **demand side** an agent pays per request (via the HTTP-native x402 micropayment flow) with no human in the loop; on the **supply side** an agent can provision a node headlessly and earn; and on the **market-creation side** an agent can launch a new cluster and its token, standing up an entire compute market programmatically. Where the human path is a multi-step wallet-signed wizard, the agent path exposes a single call returning unsigned transactions for the agent's wallet to sign (creator-pays), with fee splits and metadata taken from server-side defaults so launch is unattended.

This is the structural difference from most decentralized-compute designs, which make *participation* permissionless but keep *market instantiation* privileged (a core team lists the supported models and markets). Making cluster creation itself agentic turns the system into a permissionless **market-of-markets**, in which agents both transact in and instantiate compute markets. It also introduces the sharpest attack surface in the design: cheap programmatic market creation invites wash markets (an agent spinning up a cluster and self-serving inference to inflate volume, buy-pressure, and burn), which is why market creation must be gated by a launch fee, a stake requirement, and/or per-wallet rate limits, plus reputation-weighted visibility so unproven clusters do not surface as trusted (§10).

4. Pricing: Stable Unit, Token Settlement

Let the base price of a request be fixed in USD, $p_\$$ (e.g. \$0.01). A consumer pays in the cluster token at its live market price P_t (USD per token), so the number of tokens paid is

$$q_t = p_\$ / P_t.$$

This decouples *operator revenue* from token volatility: the node earns $p_{\$}$ of value regardless of P_t , which is essential: pricing a request at a flat token quantity would randomize operator pay as P_t moves and drive providers away.

What a rising token price does, and does not, do. We are precise here because the point is easy to overstate. Because a request is priced in USD and settled in the token, the value a user pays is $q_t \cdot P_t = p_{\$}$ — invariant in P_t . As P_t rises the request is denominated in fewer tokens (q_t falls), but each is worth proportionally more: this is a change of unit, like a stock split, not a discount to the payer. The party whose effective cost falls is a *holder* whose tokens appreciated — a capital gain (with symmetric downside), distinct from the price of the service. We keep this distinction sharp on purpose: it is what makes the network’s core claim defensible. The token does not make inference cheaper by fiat; it makes the network *self-growing* — trading activity capitalizes the Hardware Fund, which adds real machines, and it is *that* added capacity and the usage-scaled burn (§7), not token appreciation, that drive genuine cost reduction over time.

Dual-lane price schedule. Rather than forcing token usage, the protocol runs two coexisting payment lanes with a deliberate price differential. The *token lane* (pay in the cluster token) is priced at the market rate, $1.0\times$ the operator’s reference base cost B . The *stablecoin/credits lane* (pay in USDC or a prepaid USD balance) is priced at $1.5\times$, i.e. a $0.5B$ premium. In both lanes the operator receives $0.8B$ and the protocol engine $0.2B$; in the stablecoin lane the additional $0.5B$ premium is used to **buy the cluster token on the open market and burn it**. This yields three properties: (i)~**demand pull**: the token lane is $0.5/1.5 \approx 33\%$ cheaper, so rational buyers acquire and spend the token, creating buy-pressure proportional to real inference demand; (ii)~**a usage-scaled burn**: stablecoin-preferring buyers fund a burn that scales with inference volume, not speculation; and (iii)~**an optional on-ramp**: new users transact in USDC without first holding the token, yet still feed the token economy. To avoid front-running, the buyback-and-burn is executed in batches, TWAP-smoothed and slippage-limited, on non-deterministic timing rather than a fixed schedule.

Formally, for real usage priced at base cost B , write the operator share $\sigma = 0.8$, engine share $1 - \sigma = 0.2$, and stablecoin premium $\beta = 0.5$. Then per request:

$$\begin{aligned} \text{token lane: } & \text{pay } B \rightarrow \text{operator } \sigma B, \text{ engine } (1-\sigma)B, \text{ burn } 0; \\ \text{stablecoin lane: } & \text{pay } (1+\beta)B \rightarrow \text{operator } \sigma B, \text{ engine } (1-\sigma)B, \text{ burn } \beta B. \end{aligned}$$

The operator earns the same σB in both lanes, so serving is lane-neutral; the buyer, however, sees a $\beta/(1+\beta) \approx 33\%$ discount on the token lane. If a fraction s of total volume flows through the stablecoin lane, the cumulative token bought and burned is $\beta s B_{\text{total}}$, a sink that scales with inference *usage*, not speculation. Figure 1 shows both the per-request split and the usage-scaled burn.

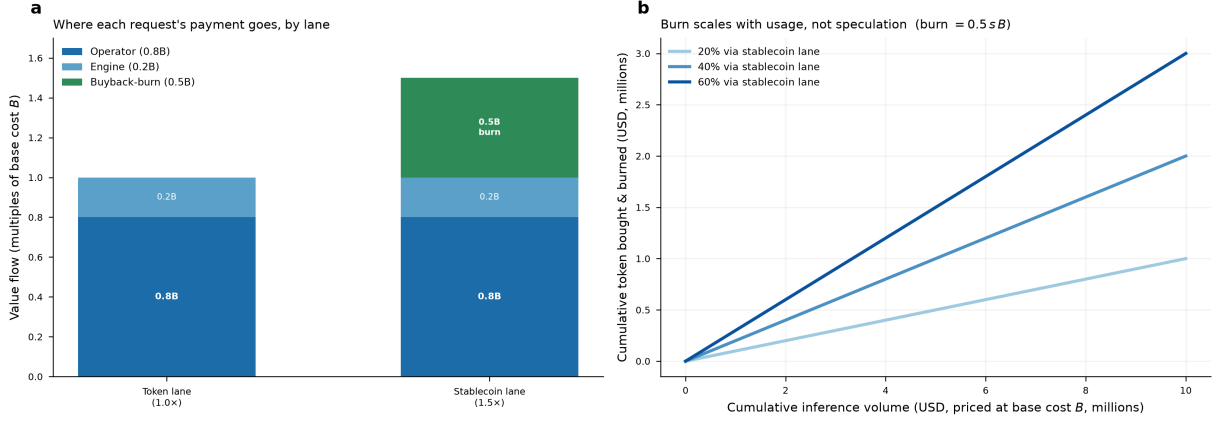


Figure 1. Dual-lane economics. **(a)** Destination of each request's payment by lane: operators receive $0.8B$ in both; the stablecoin lane's $0.5B$ premium is bought-and-burned. **(b)** Cumulative buyback-burn vs. inference volume for several stablecoin-lane shares s , the burn rate tracks real usage.

5. Security Model: Verifiable Integrity and Hardware Confidentiality

The grid provides two distinct, complementary guarantees, each enforced by the mechanism that can actually deliver it: *integrity* by verification and slashing on every node, and *confidentiality* by TEE hardware attestation on the confidential tier.

Verifiable orchestration: no trusted operator. Before either guarantee applies, one must trust *who* served a request. We remove the orchestrator from the trust base. Every node holds its own keypair and signs the assignments it accepts and the results it returns, and the protocol anchors these signatures, together with the node's staking record, on a Solana program. This design lets any party — consumer, cluster owner, or auditor — verify, without trusting Singularity's orchestrator, that a given request was routed to and served by a node that is legitimately staked, and attribute a faulty result to a specific bonded node for slashing. Orchestration policy (round-robin fairness, reputation weighting) decides *routing*; it cannot forge *provenance*.

Integrity: audit re-execution plus slashing. A provider that returns a corrupted or substituted result must be made detectable. Inference is deterministic given (model, input, sampling seed), but only *within a fixed hardware/software class*: across different GPUs, batch sizes, and kernel or runtime builds, floating-point rounding diverges, so outputs are compared within a numerical tolerance on a pinned node-class (or, for exactness-critical work, on identical node hardware), not bit-for-bit across the whole grid. On this basis the protocol audits a random fraction of jobs by re-executing them on an independent node of the same class; a mismatch beyond tolerance slashes the offending provider's staked bond S , redistributed to honest stakers. Because auditing every job would double cost, audits are sampled, so the deterrent is economic rather than absolute: with audit probability p , slashable stake S , and per-job gain-from-cheating g , honesty dominates whenever $p \cdot S > g$ — the protocol sets p (and the stake floor) to keep that inequality true for the cluster's job values, and raises p for higher-value jobs and lower-reputation providers. This is the core, and correct, use of slashing.

Confidentiality is delivered by hardware attestation. Providers join the grid two ways: with everyday consumer devices, and with *confidential cloud and data-center hardware* — TEE-capable GPUs (H100/H200 confidential computing) and CPUs (Intel TDX, AMD SEV). On the confidential tier, a TEE's remote attestation cryptographically proves the operator is excluded from the enclave, so prompts and model weights stay private even from the machine's own owner. This is a hardware guarantee, stronger than any promise, and it is exactly what regulated and sensitive

workloads require. Integrity on this tier cannot rely on audit re-execution — a confidential job cannot be re-run on another node without the plaintext input the enclave exists to hide — so here integrity rests on the *attestation* itself: the enclave proves it loaded the audited code and model hash and ran unmodified, which is what makes its output trustworthy without a second execution. Recompute-based auditing is thus the open tier's integrity mechanism; attestation is the confidential tier's. We are deliberate about scope: the open consumer tier is for public and non-sensitive workloads and does not claim enclave confidentiality — so the grid never advertises a privacy guarantee the hardware can't back, and routes sensitive jobs to attested TEE nodes. Transport is protected on both tiers: prompts and results are end-to-end encrypted client→node in transit, so no third party on the wire sees plaintext; enclave confidentiality is the stronger, separate guarantee that the serving node's own operator cannot see the plaintext either, and only the TEE tier provides it. Reputation (not stake) governs soft quality — latency and serving quality — through job-priority ranking.

Two ways to join, one integrity standard. This yields a natural tiering (Table 1): an *Open* tier on consumer hardware — near-zero marginal cost, ideal for public/batch workloads — and a *Confidential* tier on attested TEE GPUs for regulated or sensitive workloads. Both are integrity-verified — the open tier by sampled audit re-execution and slashing, the confidential tier by enclave attestation — and the confidential tier adds hardware-enforced privacy. The Hardware Fund can capitalize either, so as providers and the fund add TEE machines, the grid's community-owned confidential capacity grows.

Tier	Hardware	Confidentiality	Verifiability	Slashing enforces
Open	Consumer devices	Public / non-sensitive workloads	Sampled audit re-execution (tolerance)	Integrity (tampering)
Confidential	TEE cloud / GPU (H100/H200, TDX, SEV)	Hardware-enforced (attested)	Enclave attestation (code + model hash)	Integrity + attestation failure

Table 1. Trust tiers. Integrity is verifiable and slashable on both; confidentiality is a hardware property delivered by the TEE tier.

6. Settlement and Metering

A tokenized inference market faces a hard systems constraint: per-request charges are on the order of 10^{-3} – 10^{-2} , comparable to a single Solana transaction fee. Settling every request on-chain would spend a majority of operator revenue on fees. The mechanism therefore **meters usage off-chain and batches settlement on-chain**, while preserving custody safety, confidentiality, and verifiability. We summarize a reference implementation of the mechanism; details marked as implementation choices are not fundamental to the mechanism.

6.1 Reserve → Settle → Release

Because token-by-token streaming can only bill after generation completes (the token count is unknown until then), the mechanism debits a *pre-funded, raw-token-denominated balance* through a three-phase protocol. Balances are held in raw token units, not a locked USD value, so as the token price rises the same balance buys more inference. (1)~**Reserve**: at submission the coordinator estimates the request's maximum cost, converts it to raw token at the current oracle price, and, under a row lock on the balance, reserves that amount, rejecting with HTTP 402 *before* any node is dispatched if funds are insufficient (nothing is charged). (2)~**Serve**: the sealed prompt

is dispatched to the reserved node. (3)~**Settle**: on completion the coordinator prices the *actual* usage, clamps it to the reservation, debits the balance, and accrues the operator's 80% share; on any failure (timeout, empty completion, node error) it **releases** the reservation and the buyer spends nothing. A periodic sweeper settles or releases any reservation orphaned by a crash. Correctness rests on database-level invariants: atomic row-locked balance functions, compare-and-set job transitions that make double-settlement structurally impossible, and single-use idempotency gates keyed by job identity so that retries, REST/WebSocket duplication, and reaper races cannot double-charge or double-pay.

Concretely, with token decimals d and oracle price π (USD per token) at reserve time, the reserved raw amount for a request quoted at q_{usd} (its maximum cost) is

$$\text{reserve_raw} = \lceil q_{\text{usd}} \cdot 10^d / \pi \rceil,$$

and at completion the actual charge is priced on real usage *and clamped to the reservation*, so the buyer can never be charged above the quote they accepted:

$$\text{actual_raw} = \min(\lceil q_{\text{usd}}^{\text{actual}} \cdot 10^d / \pi \rceil, \text{reserve_raw}).$$

The operator accrues a share $\sigma = 0.8$ of the actual charge; the protocol engine retains $1 - \sigma = 0.2$. Because the completion-token count is capped to the request's own `max_tokens`, a node that overstates usage cannot drain more than the buyer's prepaid quote: the worst-case loss equals the amount already agreed (§5).

6.2 Epoch-batched payouts via a cumulative Merkle distributor

Operator earnings accrue in an off-chain ledger and are paid out through an on-chain **cumulative Merkle distributor** (one long-lived contract per token). Once per epoch the coordinator publishes a Merkle root whose leaves are each claimant's all-time cumulative total; an operator claims the difference between their cumulative total and what they have already claimed, sweeping *all* accrued earnings in a single transaction at any time. This converts an $O(\text{operators} \times \text{requests})$ on-chain settlement cost into $O(\text{epochs})$, a flat, per-epoch root update regardless of operator count, with operators self-claiming and paying their own (small) fee. Operators are paid **in the currency the buyer paid** (pay-in-kind): the token lane is a treasury pass-through with no swap; the only protocol swap is the stablecoin-lane buyback-and-burn (§4), minimizing slippage and MEV surface on the cluster token.

The economic necessity of batching is stark. For N requests each earning an operator $r \approx \$1.6 \times 10^{-3}$ and an on-chain settlement costing $c_{\text{tx}} \approx \$10^{-3}$, naïve per-request settlement consumes a fixed fraction $c_{\text{tx}}/r \approx 62\%$ of operator revenue, independent of scale. The epoch-batched model costs one root update per epoch, so its share of revenue is

$$\text{cost fraction} = c_{\text{root}} / (N \cdot r) \rightarrow 0 \text{ as } N \text{ grows},$$

turning an $O(\text{operators} \times \text{requests})$ on-chain cost into $O(\text{epochs})$. Figure 2 plots both.

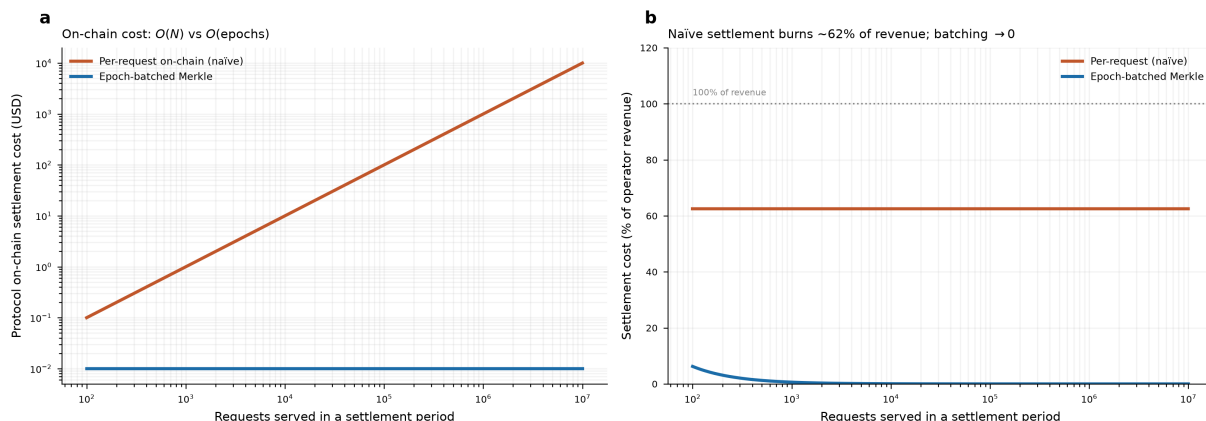


Figure 2. Why settlement must be batched. **(a)** Protocol on-chain settlement cost vs. requests served: per-request settlement grows linearly, epoch-batched Merkle is flat. **(b)** As a fraction of operator revenue, naïve settlement burns a constant ~62%, while batched settlement tends to zero as volume grows.

7. Incentive Analysis

Token value and velocity. A pure medium-of-exchange token is subject to the velocity critique: [11] if users buy just-in-time and node operators immediately sell to cover real-world costs, buy and sell pressure cancel and usage alone does not support price. TCM introduces structural holding demand and sinks that counteract velocity:

- **Staking lock.** Each provider locks a fixed bond S of the base token, removing supply from circulation.
- **Swap-fee levy.** A levy τ on swaps through the cluster's canonical pool continuously diverts value to the Hardware Fund (infrastructure) and/or node providers, so trading, not merely holding, funds real capacity.
- **Buyback-and-burn.** The USDC premium and a protocol share fund buybacks and burns, tightening supply as usage grows.

The levy is the key coupling: it converts market turnover into a usage/liquidity-correlated revenue stream for physical infrastructure, independent of whether any individual holds.

A usage-linked sink. To tie token value to *consumption* rather than speculation, a fixed fraction of every inference fee (not only the USDC premium) can be routed to buyback-and-burn, so that supply tightens monotonically with real usage. We recommend this as a design default.

Reconciling turnover with the velocity critique. There is an apparent tension between this section and the Hardware Fund (§8): the fund is financed by trading turnover, yet turnover *is* velocity, the very thing that undermines price support. We treat this directly. The two are reconciled by *what kind* of turnover is rewarded. Raw swap volume is not the target — if it were, a cluster owner would be incentivized to wash-trade their own token, since self-trading would inflate volume, buy-pressure, burn, and hardware-target hits at will. The mechanism therefore gates fund accrual and reputation on *externally-paid* activity, netting out the owner's own wallets, so the levy converts *genuine* demand-driven turnover into capacity while the staking lock, usage-linked burn, and pre-committed parameters counteract purely speculative velocity. Turnover from real usage is the fuel; speculative churn and self-dealing are screened out rather than subsidized.

Cluster competition. Clusters compete on models offered, price, reliability, and forthcoming capacity. Because switching is cheap, this competition disciplines quality and pushes effective prices toward marginal cost, a benefit to consumers and a selection pressure favoring well-run

clusters. A further capability, deferred to a companion paper, is *intra-cluster model sharding*: because a cluster is a coordinated set of machines under one economic umbrella, a model too large for any single node can be sharded across the cluster's members, letting community-owned hardware serve models that no individual consumer device could host. We note this only as a direction here; its scheduling, latency, and fault-tolerance analysis is future work.

Why owners route a meaningful fraction. The levy τ and routed fraction ρ are owner-set, which invites the question of why an owner would not choose $\tau \rightarrow 0$ or $\rho = 0$ to maximize short-term extraction. The constraint is competitive, not enforced: a cluster with $\rho = 0$ accrues nothing to its Hardware Fund, never adds machines, and therefore cannot make the self-growing-capacity promise that attracts token buyers and node operators in the first place: its token has no capacity narrative and its supply cannot grow to meet demand. Because clusters compete for both users and providers, and because these parameters are fixed transparently before token launch (they cannot be changed to rug participants afterward), the market selects for owners who route enough to fund visible growth. The equilibrium is thus a positive ρ disciplined by competition and by pre-commitment, not a mandated minimum; formalizing the owner's optimal (τ, ρ) under competition is left to future work.

8. Illustrative Capacity Model

We illustrate the Hardware Fund with transparent, clearly-labelled parameters (not measurements). The fund's fee base is *trading volume*, not market capitalization: for cumulative traded volume V , the fund accrues

$$F(V) = \tau \cdot \rho \cdot V,$$

which is exact: no assumption about how volume relates to price. The number of machines of unit cost C_h the fund can add is $n_h(V) = F(V) / C_h$. Figure 3 plots F and n_h against V for $\tau = 0.02$ (the 2% transfer levy) and $\rho = 0.5$ (half the levy routed to the fund). A market-cap figure only enters through a separate *turnover* assumption $V = \kappa M$; at an illustrative $\kappa = 3$, a \$1B market cap corresponds to \$3B cumulative volume (dotted line), yielding a ~\$30M fund, enough for ~15,000 consumer nodes or ~865 owned H100-class GPUs. We plot against volume precisely so this turnover assumption is visible and separable, rather than baked into the axis.

Sensitivity to turnover. Because the fund is linear in volume, the market-cap-implied figure scales directly with κ . Holding a \$1B market cap fixed: at $\kappa = 1$ (volume equals market cap once) the fund is ~\$10M (~5,000 nodes or ~285 owned H100s); at $\kappa = 3$ it is ~\$30M (~15,000 / ~855); at $\kappa = 10$ it is ~\$100M (~50,000 / ~2,855). Empirically, annualized turnover for actively traded small-cap tokens spans a wide range, so κ is best read as a scenario parameter rather than a point estimate; the mechanism's validity does not depend on any particular value, only on κ and the routed fraction ρ being strictly positive.

Hardware Fund vs cumulative trading volume (the actual fee base)

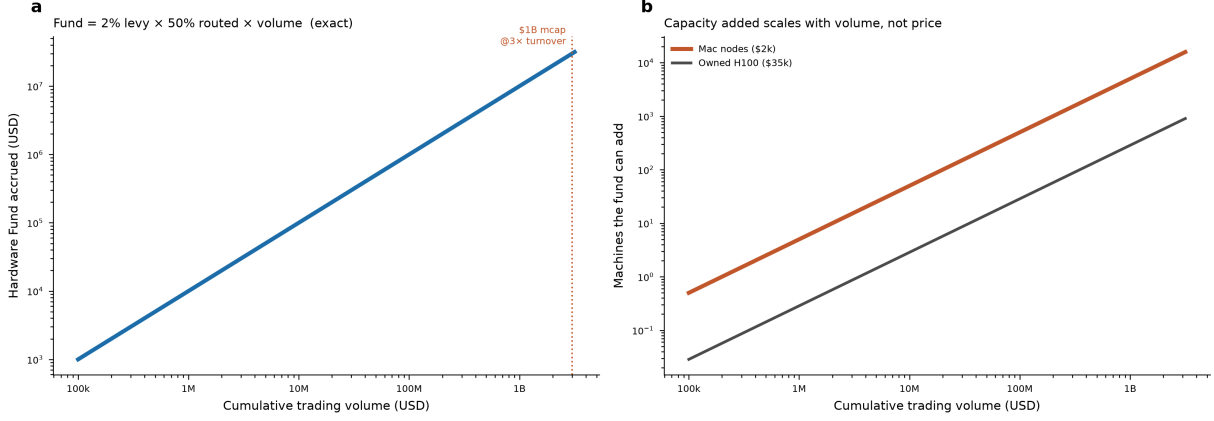


Figure 3. Hardware Fund vs. cumulative trading volume, the actual fee base. **(a)** Fund balance $F = \tau\rho V$ (exact). **(b)** Machines addable. The dotted line marks the volume (\$3B) implied by a \$1B market cap at 3× turnover; this is the only place a price assumption enters. All values illustrative.

The point is the *coupling*: capacity scales with realized trading *volume* (liquidity and usage) and the owner's routing choice ρ trades immediate node-provider yield against long-run physical growth.

8.1 Pricing in the cluster token

Figure 4 makes the pricing rule of §4 concrete. As the token price rises, the *number* of tokens a user pays for a fixed \$0.01 request falls ($100 \rightarrow 10 \rightarrow 1$), which is often described as “cheaper inference.” Panel (b) is the necessary companion: the *value* paid is flat at exactly \$0.01 throughout. Fewer tokens, each proportionally more valuable, is a change of unit, not a discount for the paying user.

Paying in the cluster token: fewer tokens as price rises, same real cost

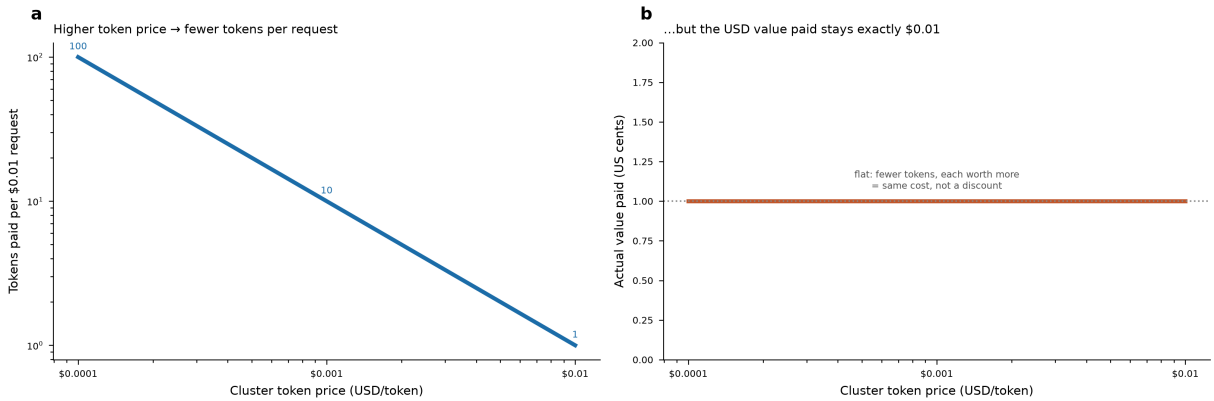


Figure 4. Settling inference in the cluster token. **(a)** Tokens paid per \$0.01 request vs. token price. **(b)** The USD value paid is invariant at \$0.01. The two panels together prevent the “higher price = cheaper inference” misreading (§4).

8.2 Self-growing throughput

Because the fund converts volume into machines, and machines into serving capacity, the grid's aggregate throughput grows with trading volume. Figure 5 illustrates the throughput (text tokens per second) the fund unlocks on each tier (using illustrative per-node throughputs). This is the “self-growing compute” property: the network enlarges its own capacity as its market is used, subject to the demand constraint of §10.

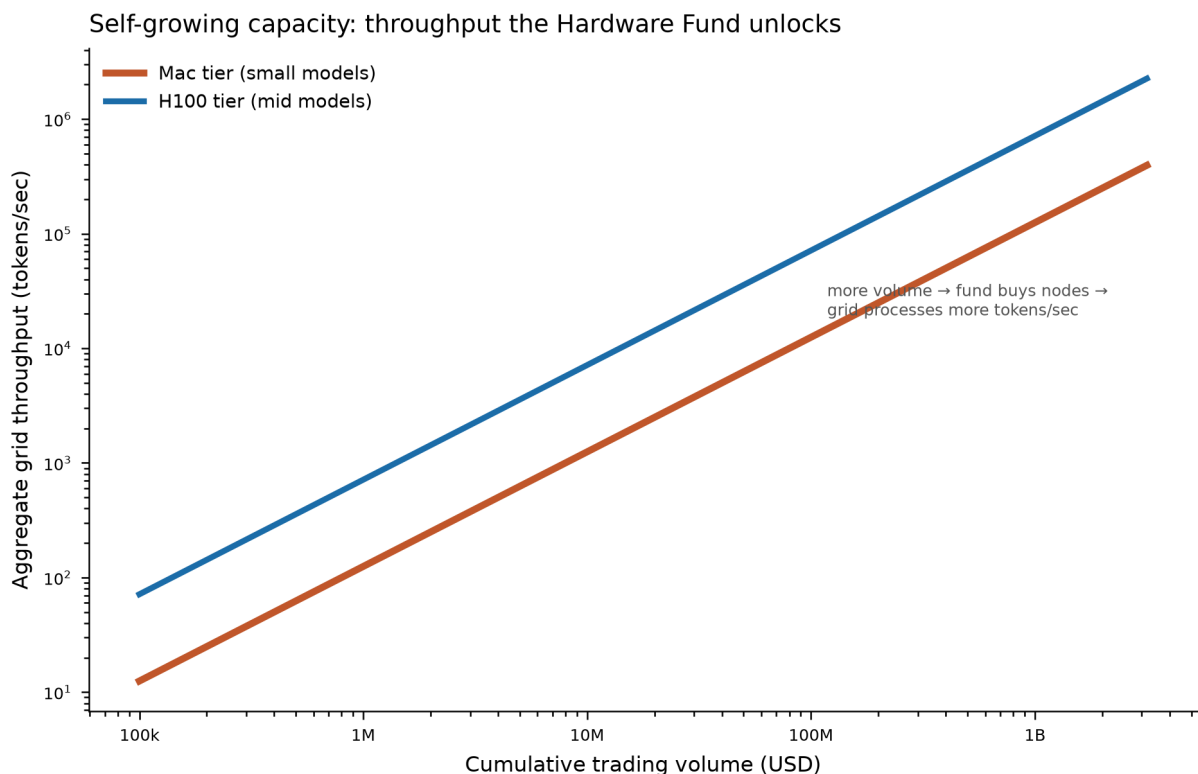


Figure 5. Aggregate grid throughput unlocked by the Hardware Fund, vs. cumulative trading volume, for the Mac tier (small models) and H100 tier (mid-size models). Per-node throughputs are illustrative; the shape (capacity rising with volume) is the structural claim.

9. Real-World Comparison: Manual Buildout vs. Self-Growing Fund

We now compare the conventional way a community or company would deploy AI compute (buying it) against the TCM Hardware Fund path, using current (2025–26) market prices. **An essential clarification first:** TCM does *not* make the hardware cheaper. An H100-class GPU costs \$25,000–\$40,000 whether an owner buys it directly or the fund buys it on the cluster's behalf. What TCM changes is the *capital structure*: **who fronts the money, when, and who carries the utilization risk**, not the unit price. Any claim of a per-unit cost saving would be false.

The manual path. To stand up an owned cluster, an operator faces a large day-zero capital outlay and continuing operating cost. Representative figures: a complete 8-GPU H100 node runs \$200,000–\$320,000 including server, networking, and infrastructure, with 6–12 month procurement lead times; a 256-GPU pod is \$7–10M; and a purpose-built AI datacenter runs \$20M+ per megawatt of capacity, so a 50 MW facility exceeds \$1B in construction alone, before a single GPU is installed. On top of capex sit power, cooling, colocation (\$170–215/kW/month), maintenance, and staff (one DevOps engineer per 50–100 GPUs at \$120,000–\$200,000/yr). Crucially, the operator bears all of this *before* knowing whether demand will materialize, the classic utilization risk that makes ownership uneconomic below ~60–70% sustained use.

The self-growing path. Under TCM, the cluster owner fronts essentially none of that capital. The Hardware Fund accrues from the transfer levy on trading and acquires machines incrementally as targets are hit, each machine added only once the community's market activity has already paid for it. Capacity therefore grows in step with the market's own scale, and the owner never signs a \$10M capex cheque against uncertain demand. Table 2 and Figure 6 make the contrast concrete.

Cluster size	Manual: owner upfront	Manual: annual opex	TCM: owner upfront	TCM: volume the levy must accrue
8 GPUs (1 node)	~\$0.3M	~\$0.1M/yr	~\$0	~\$30M cumulative
64 GPUs (8 nodes)	~\$2.4M	~\$0.8M/yr	~\$0	~\$240M cumulative
256 GPUs (32 nodes)	~\$9.6M	~\$3.2M/yr	~\$0	~\$960M cumulative

Table 2. Manual buildout vs. Hardware Fund, at 2025–26 prices (\$300k per 8-GPU node all-in; \$100k/node/yr opex; 2% levy with 50% routed to the fund). The TCM column moves the cost from the owner's balance sheet onto cumulative market volume; it does not eliminate it.

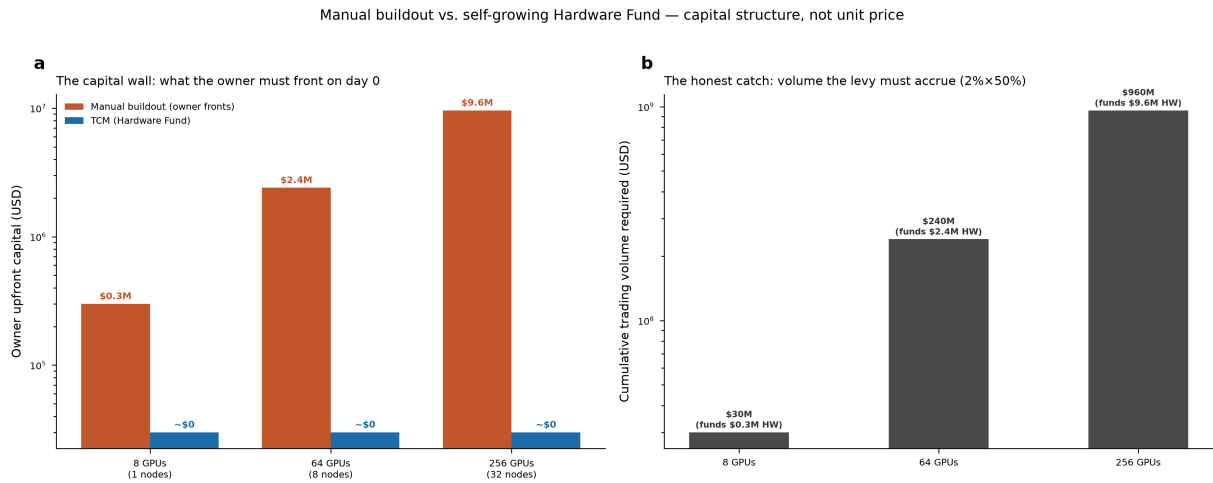


Figure 6. Capital structure, not unit price. **(a)** Day-zero capital the owner must front: millions for a manual buildout vs. ~\$0 under the Hardware Fund. **(b)** The honest cost of that shift: the cumulative trading volume the 2%×50% levy must accrue to fund the same hardware. TCM removes the capital wall but substitutes a volume requirement.

The honest trade-off. The self-growing path is not free money: it relocates the cost. A manual buildout demands capital the owner may not have; the TCM path demands *trading volume the market may not generate*. Funding a 256-GPU pod purely from a 2%×50% levy requires roughly \$960M in cumulative volume, which only a genuinely active market produces. This is the same demand constraint as §10: the fund scales real capacity only when real economic activity flows through the cluster. The advantage TCM offers is not cheaper hardware but *de-risked, demand-following capital formation*: the community funds capacity it is already using, incrementally, without a central operator staking millions against a forecast.

9.1 One-click deployment

To make supply-side participation frictionless, a provider (human or agent) can deploy through a single guided flow: select GPU type, select model, deploy, and the node is attested (where applicable) and joined to the grid automatically. This lowers the barrier to contributing capacity from a datacenter-engineering task to a consumer-grade action, widening the pool of potential node providers and feeding the same self-growing loop from the supply side.

10. Failure Modes and Design Constraints

F1: Supply without demand (reflexivity). The loop mechanizes *supply* growth (trades → machines) but not *demand*. If trading is speculative rather than driven by real inference consumption, the fund deploys idle capacity; when speculation fades, the loop unwinds, the

canonical DePIN failure. **Constraint:** demand must lead supply. A cluster should demonstrate real external paying usage before the Hardware Fund scales capacity, and the protocol should weight incentives toward externally-paid inference rather than self-generated keep-alive traffic. Listing grid endpoints on existing inference marketplaces (OpenAI-compatible routers) is a low-cost way to source genuine demand signal.

F2: Velocity. Addressed in §7 via the staking lock, transfer levy, and usage-linked burn; without at least one usage-correlated sink, the token price is unsupported by usage.

F3: Liquidity fragmentation. Per-cluster tokens split demand and liquidity across many thin markets, and a consumer seeking a given model should not have to hold the “right” cluster’s token. **Constraint:** inference demand should route to the best-matching node globally, with the cluster token serving as the ownership/revenue layer rather than a hard access wall; otherwise clusters risk becoming illiquid and models become stranded behind whichever token is tradeable.

F4: Confidentiality overclaim. Promising slashing-enforced confidentiality on non-TEE hardware is unfulfillable (§5). **Constraint:** confidentiality is claimed only for attested TEE nodes; consumer nodes are marketed for non-confidential workloads.

F5: Sybil and wash markets. Because inference volume drives token demand and burn (§4), and because market creation is permissionless and agentic (§3.1), an operator can spin up a cluster and self-serve inference to inflate volume, buy-pressure, and burn. The sharp point is that this is not merely an external-attacker risk: because the Hardware Fund and burn are financed by turnover, the cluster *owner* is the party with the strongest wash incentive, since self-trading directly advances their own hardware targets. **Constraint:** the load-bearing fix is to make the fund and reputation accrue on *externally-paid* inference and swap fees *net of the owner’s own wallets*, so self-trades add nothing to fund, burn-credit, or visibility; this is layered with economic friction on participation and creation (a launch fee, a stake requirement, per-wallet rate limits) and reputation-weighted visibility so unproven clusters are not surfaced as trusted. Netting depends on identifying owner-linked wallets, which is imperfect under Sybil pressure; formalizing a fully incentive-compatible, wash-resistant version is an open problem, but gating on external revenue removes the first-order self-dealing incentive rather than leaving it unpriced.

F6: Regulatory surface. Transfer-levy tokens and buyback-burn mechanics interact with exchange listing and securities treatment; the mechanism should be reviewed with competent legal counsel before deployment. (This paper is not legal advice.)

11. Conclusion

Tokenized Compute Markets couple capital formation, infrastructure deployment, and inference into one loop, so that community trading finances community-owned compute. The design’s strength is the Hardware Fund’s direct conversion of market activity into physical capacity; its discipline is a strict separation of what economics can enforce (integrity) from what only hardware can (confidentiality), and an honest account of pricing that resists selling token appreciation as a service discount. The mechanism is sound insofar as demand leads supply, at least one usage-correlated sink counteracts velocity, and confidentiality is claimed only where hardware can guarantee it. We release this as a design paper ahead of deployment to invite scrutiny of these mechanics.

References

1. J. X. Morris, V. Kuleshov, V. Shmatikov, A. M. Rush. “Text Embeddings Reveal (Almost) As Much As Text.” arXiv:2310.06816, 2023.
2. L. De Castro, D. Escudero, A. Agrawal, A. Polychroniadou, M. Veloso. “EncryptedLLM: Privacy-Preserving Large Language Model Inference via GPU-Accelerated Fully Homomorphic Encryption.” Proc. 42nd Int. Conf. on Machine Learning (ICML), PMLR 267:12677–12688, 2025.
3. Y. Dong, W.-J. Lu, Y. Zheng, et al. “PUMA: Secure Inference of LLaMA-7B in Five Minutes.” arXiv:2307.12533, 2023.
4. M. Yuan, L. Zhang, X.-Y. Li. “Secure Transformer Inference Protocol.” arXiv:2312.00025, 2023.
5. F. Zheng, C. Chen, Z. Han, X. Zheng. “PermLLM: Private Inference of Large Language Models within 3 Seconds under WAN.” arXiv:2405.18744, 2024.
6. R. Thomas, L. Zahran, E. Choi, A. Potti, M. Goldblum, A. Pal. “An Attack to Break Permutation-Based Private Third-Party Inference Schemes for LLMs.” arXiv:2505.18332, 2025.
7. Y. Li, X. Zhou, Y. Wang, L. Qian, J. Zhao. “A Survey on Private Transformer Inference.” arXiv:2412.08145, 2024.
8. M. Ugurbil, D. Mouris, M. B. Santos, J. Cabrero-Holgueras, M. de Vega, S. Sengupta. “Fission: Distributed Privacy-Preserving Large Language Model Inference.” IACR Cryptology ePrint Archive, Report 2025/653, 2025.
9. J. Zhu, H. Yin, P. Deng, A. Almeida, S. Zhou. “Confidential Computing on NVIDIA Hopper GPUs: A Performance Benchmark Study.” arXiv:2409.03992, 2024.
10. M. Chrapek, M. Copik, E. Mettaz, T. Hoefler. “Confidential LLM Inference: Performance and Cost Across CPU and GPU TEEs.” arXiv:2509.18886, 2025.
11. V. Buterin. “On Medium-of-Exchange Token Valuations.” 2017.
12. Gartner worldwide IaaS public cloud market figures for 2024 (market up 22.5% to \$171.8B; the three largest hyperscalers — AWS (~38%), Microsoft Azure (24%), Google Cloud (9%) — together near 71%), as reported in M. Ashare, “Cloud’s big 3 continue to rule infrastructure services,” *CIO Dive*, 11 August 2025.